

Dual-UAV Vision-Based Inspection of Oil Pipelines Using Line Tracking and YOLO-Based Anomaly Detection

Álvaro Aguirre Castillo (A00836166), Joel Aragón Balderas (A01285754),
Emilio Carrillo Zardain (A01749795), Carlos Cuadras Santaella (A01742709),
Iván Enrique Rodríguez Estrada (A00834880)
Department of Mechatronics Engineering
Tecnológico de Monterrey
Email: first.author@tec.mx

Abstract—This report presents a dual-unmanned aerial vehicle (UAV) framework for visual inspection of oil pipelines using low-cost DJI Tello drones and a deep-learning based perception module. The first UAV performs autonomous line tracking over a white line representing the pipeline, detects visual anomalies with an Ultralytics YOLO model, and classifies inspection points as safe or defective based on the presence of fire. A second UAV then executes a waypoint-based mission that revisits only the defective segments for detailed re-inspection. The system integrates real-time computer vision, multi-phase mission logic, PID-based visual servoing, and a custom-trained YOLO detector for fire, anomalies, and human shoes, used as a safety proxy for personnel presence. Mission outcomes are logged in a text-based segment file and used to plan the second UAV's route. The proposed approach shows how commodity drones, open-source libraries, and lightweight neural networks can be combined to create a low-cost pipeline inspection prototype suitable for education and research.

Index Terms—UAV, oil pipeline inspection, YOLO, computer vision, multi-UAV, line tracking, DJI Tello.

I. INTRODUCTION

Oil and gas pipeline inspection is traditionally performed with ground patrols, manned vehicles, or specialized in-line inspection tools, approaches that tend to be costly and may expose personnel to hazardous environments. Unmanned aerial vehicles offer an attractive alternative by providing low-altitude visual inspection with reduced operational risk and cost. Multi-UAV systems can further reduce inspection time by distributing tasks among vehicles, for example assigning one UAV to perform fast coverage and another to conduct detailed re-inspection of suspicious regions [1], [2]. In parallel, deep learning-based object detection has become the de facto standard for recognizing safety-relevant events in images, such as fire, leaks, and human presence, and the Ultralytics YOLO family provides state-of-the-art real-time detectors that are straightforward to train and deploy on edge devices [3], [4].

The work presented here describes a complete prototype of a dual-UAV inspection system implemented with two DJI Tello drones, the `djitellopy` Python library, OpenCV, and a custom-trained Ultralytics YOLO model. The first drone performs continuous patrol and autonomous line tracking

along a white strip that represents the pipeline, while the second drone is deployed after the first mission finishes and revisits defective points identified by the perception module. The complete behavior is encoded in a single Python program (`reto_final.py`), which integrates perception, control, mission logic, WiFi management and communication with an external Arduino. The main contributions of this work are the definition of a dual-UAV inspection architecture where a patrol UAV logs discrete inspection points and a second UAV revisits only defective segments, the implementation of a vision-based line tracking and inspection controller on a low-cost drone platform, and the design and training of a custom YOLO pipeline for detecting fire, generic anomalies, and human shoes that is integrated into the flight control loop.

The remainder of the report is organized as follows. Section II describes the system architecture, Section III focuses on the perception and computer vision modules, Section IV presents the autonomous mission for Drone 1, Section V details the mission logic for Drone 2, Section VI explains the YOLO training process, Section VII sketches an experimental protocol, Section VIII discusses limitations and future work, and Section IX concludes.

II. SYSTEM ARCHITECTURE

A. Hardware Platform

The prototype relies on two DJI Tello micro-drones controlled by the `djitellopy` Python wrapper around the official Tello SDK [5]. Each drone provides an RGB video stream over WiFi and uses its internal stabilization, without any external motion capture system. A laptop acts as a ground station running Python, OpenCV, the Ultralytics YOLO library and `pywifi` for WiFi network management. An Arduino board is connected over a serial link at 115500 baud, and the Python script sends single-byte codes to this board to trigger external actuators, such as LEDs or buzzers, depending on the detection results produced by the UAV perception module.



Figure 1: Drone Tello SDK

B. Software Components

The main software blocks implemented in `reto_final.py` are a video acquisition thread, a perception module, a control and mission logic module, a WiFi management module and a logging module. The video acquisition thread opens the Tello video stream `udp://0.0.0.0:11111`, continuously reads frames, resizes them to 640×480 pixels and updates a global frame buffer variable. This decouples the perception and control logic from the raw video capture and ensures that the rest of the system always processes the latest available frame.

The perception module combines white line detection using HSV thresholding and contour analysis with a YOLO-based detector that identifies fire, anomalies and shoes using a custom `best.pt` model. The control and mission logic implement the state machines for both drones, where Drone 1 performs line tracking, anomaly-triggered inspection and safety pauses, and Drone 2 executes waypoint navigation for re-inspection based on previously logged defective points. The WiFi management module uses `pywifi` to connect sequentially to each drone's SSID and to switch between networks as the missions progress. The logging module writes segment information and inspection results to a text file named `traj_segments.txt` at the end of the first mission and later reads the same file to plan the route for the second UAV.

C. Functional Roles of the Two Drones

Drone 1 is responsible for patrol and detection. It follows the pipeline using visual line tracking, segments the path into discrete inspection points and classifies each point as safe (BUENO) or faulty (MALO) based on fire detections during local inspections. Drone 2 is responsible for targeted re-inspection. It executes a discrete path over the pipeline in a global coordinate frame, visiting only those points that Drone 1 marked as defective. The second mission thus focuses the available flight time on regions of interest rather than repeating a full coverage of the duct.

III. PERCEPTION AND COMPUTER VISION

A. Video Acquisition

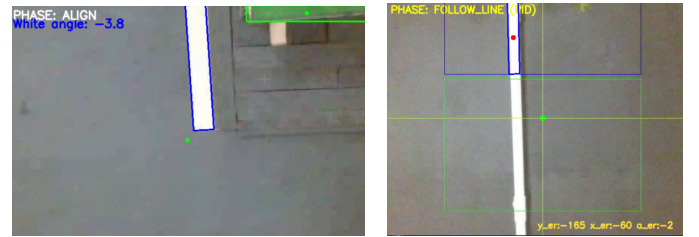
The video acquisition function uses OpenCV's `VideoCapture` interface to retrieve frames from the Tello stream. Each frame is resized and copied into a global variable that acts as a shared buffer between the acquisition

thread and the main control loop. The control loop verifies that the global frame is valid and processes it whenever a new frame is available. If the stream fails or the frame is invalid for a prolonged period, the mission logic detects the error and stops the mission in a safe manner.

B. White Line Detection

The pipeline is represented by a white line painted on the floor, and two complementary functions estimate its orientation and position. The global orientation function converts the RGB frame to HSV, applies a threshold for a narrow white range to obtain a binary mask, extracts the largest contour, approximates it with a minimum-area rectangle and computes the rectangle orientation, which approximates the direction of the pipe in the image. Extreme angles that exceed a saturation threshold are clamped to avoid spurious values, and this global orientation is primarily used during the alignment phase before line following.

The local line angle and centroid function focuses on a region of interest near the top center of the image where the line is expected to appear. It extracts a rectangular ROI, converts it to HSV, thresholds white pixels, applies morphological opening to reduce noise and finds contours in the resulting mask. Contours with area below a minimum threshold are discarded, and for the largest remaining contour the function computes a minimum-area rectangle and obtains its orientation and centroid coordinates. These outputs, together with a boolean flag indicating whether a valid line was found, are used as inputs to the visual servoing controller in the line-following state.



(a) Pipe Align phase

(b) Follow Pipe phase

Figure 2: Two phases the drone conducts in order to follow the pipe: align with angle first, then move.

C. YOLO-Based Detection of Fire, Anomalies and Shoes

The YOLO-based detection function performs inference using a custom Ultralytics YOLO model instantiated as `YOLO("best.pt")`. Inference is carried out on each frame or at a configurable rate with an image size of 320×320 pixels to reduce latency during flight. For each detection, the class index and confidence score are extracted, and detections with confidence below a given threshold are discarded. The model is trained with three classes corresponding to fire, anomaly and shoe. Bounding boxes are drawn on the frame with color-coded rectangles and labels to facilitate visualization during experiments, and the centers of the boxes are recorded to verify

whether each detection lies inside specific regions of interest that are relevant either to inspection or to safety behavior. The function returns boolean flags indicating the presence of each class, the centers of the detections and the annotated frame.



Figure 3: Model training results example

IV. AUTONOMOUS MISSION FOR DRONE 1

A. Mission State Machine

The mission logic for Drone 1 is implemented as a state machine with phases corresponding to alignment, line following, inspection and elevation search. In the alignment phase the drone uses the global orientation estimate of the pipe to rotate until its heading roughly matches the direction of the duct. In the line-following phase the UAV tracks the line while progressing along the duct by relying on the local angle and centroid provided by the line detection function and applies PID controllers to maintain its position over the pipeline. In the inspection phase the drone stops and performs local inspection when an anomaly is detected in a specific region of interest. In the elevation search phase the vehicle attempts to re-acquire the line after losing it for more than a configured memory interval, using vertical and lateral maneuvers guided by continuous visual feedback.

User inputs from the keyboard are interpreted as high-level commands. The space bar initiates takeoff, the key $\pm$ toggles between automatic and manual modes, the key $\downarrow$ commands landing and the escape key terminates the program. These commands are integrated with the state machine so that automatic behavior can be interrupted when required.

B. PID-Based Visual Servoing

In the line-following phase three errors are computed: a horizontal error given by the difference between the line centroid and the image center, a vertical or forward error related to the distance between the desired vertical position and the observed centroid, and an angular error given by the difference between the desired angle, which is zero in the camera frame, and the measured line angle. For each error signal $e(t)$, a PID controller computes a control signal of the form

$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}$. The resulting control signals are mapped to left/right velocity, forward/backward velocity and yaw rate in the Tello command space. Each control output is saturated to a maximum magnitude to avoid overly aggressive maneuvers, and a small constant forward bias is added to the forward velocity so that the UAV keeps progressing along the duct while compensating lateral and angular errors.

C. Memory Mode and Elevation Search

If the line detection function fails to find a valid line, the system first enters a memory mode in which the last valid control commands are reused with exponential decay for up to a predefined memory time. This behavior assumes that the duct continues straight and provides robustness against short occlusions, shadows or brief illumination changes. If the line is not re-acquired within the memory interval, the state machine transitions to the elevation search phase. In this phase the drone executes a vertical move to rise above its previous height and then performs a lateral search to the left or to the right while continuously running line detection. If the line is detected again, the UAV adjusts its position and returns to the line-following phase. This mechanism offers a controlled recovery strategy before the mission is aborted.

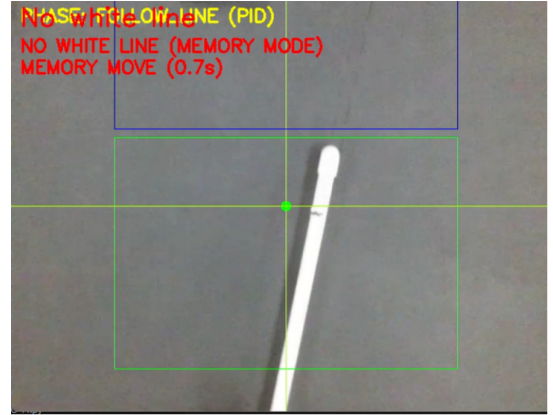


Figure 4: Visual interface while in Memory Mode.

D. Anomaly-Triggered Inspection

An anomaly detection is considered relevant when the center of its bounding box lies within a predefined green region of interest in front of the drone and a cooldown period has elapsed since the last inspection. When this condition is satisfied, the current movement segment between inspections is closed and its average forward speed and duration are stored. The drone then transitions to the inspection phase, reduces its velocity and remains nearly stationary above the anomaly location while an inspection timer is started. During this phase YOLO continues to run on each frame and, if a fire detection occurs at any moment within the inspection window, a boolean flag indicating that fire has been seen is set. Once the inspection time elapses, the inspection point is labeled as faulty if the fire flag is true or as safe otherwise,

and the corresponding code is sent to the Arduino so that external warnings can be activated if necessary. The inspection index is incremented and the state machine returns to the line-following phase until the maximum number of inspection points is reached or the user commands landing.

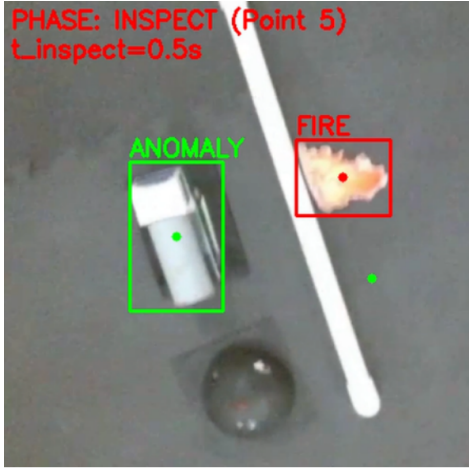


Figure 5: Visual interface when in Inspection Phase, while detecting fire.

E. Shoe-Triggered Safety Pause

Detection of a shoe is used as an indicator of human presence near the duct. If a shoe is detected outside the inspection state, the system enters a safety pause. In this pause all velocity commands are set to zero, a dedicated safety code is sent to the Arduino to trigger a human-presence warning and a visual overlay on the video shows that a shoe has been detected and the mission is paused. When the shoe is no longer detected, the safety code is reset, timers and cooldowns are adjusted to avoid corrupting inspection timing and the state machine resumes from the previous phase. This behavior is intended as a proxy for human-in-the-loop safety policies around hazardous infrastructure.

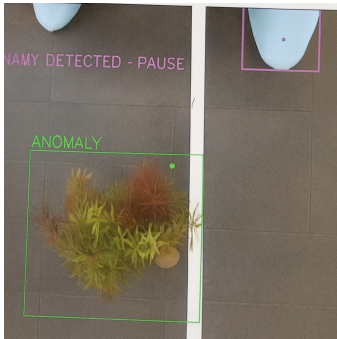
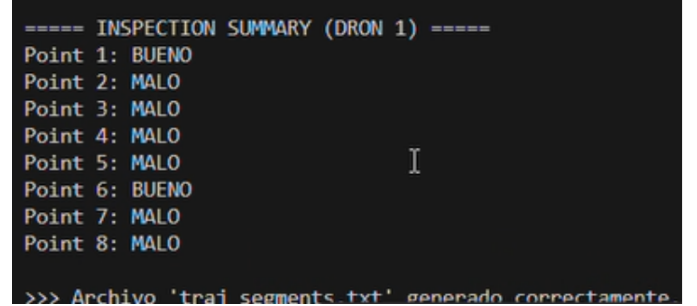


Figure 6: Visual interface while in Safety Pause.

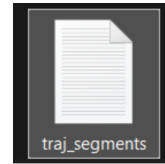
F. Logging of Segments and Inspection Results

At the end of the mission, the system writes a text file called `traj_segments.txt` that contains the information

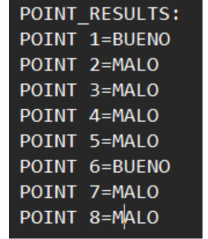
required by the second UAV. This file stores a list of segments between inspections, including the average forward speed and the duration of each segment, and may include flags that indicate discrete events such as ninety-degree turns. The file also stores a summary of inspection outcomes, where each inspection point is labeled as BUENO or MALO. This structure provides a compact representation of the patrol mission that is easy to parse and use as input for subsequent re-inspection planning.



(a) Results printed on operating PC's terminal.



(b) Created "traj_segments" text file.



(c) Contents of "traj_segments" text file.

Figure 7: Logs of inspection results.

V. MISSION LOGIC FOR DRONE 2

A. Parsing Defective Points

The second UAV begins by parsing the results of the first mission. A function scans the segment file for the section containing the point results and reads each line describing the outcome of an inspection point. From this information the function constructs a binary vector in which each entry is equal to one if the corresponding point has been labeled as MALO and zero if it has been labeled as BUENO. This vector acts as a mask that specifies which points require re-inspection by Drone 2.

B. Waypoint Generation from a Global Map

The mission for the second UAV is defined over a set of two-dimensional coordinates in centimeters that represent the inspection points along the duct in a global reference frame. These coordinates are defined based on the layout of the mock pipeline and include positions corresponding to the origin and subsequent points along rectilinear segments and corners. A list of waypoints is then constructed by starting at the origin, adding each point whose corresponding mask entry is equal to one and optionally returning to the origin at the end of the

route. This list specifies the sequence of target points that the second drone must visit.

C. Relative Motion and Command Translation

For each pair of consecutive waypoints, the mission logic computes the displacement in the global frame as the difference between the target and the current coordinates. The heading angle associated with this displacement is obtained using the arctangent of the displacement components and converted from radians to degrees. This heading is then expressed as a relative rotation with respect to the last commanded yaw by subtracting the previous heading and normalizing the difference to the range between minus 180 degrees and plus 180 degrees. The Euclidean distance between the two waypoints is also computed and, based on this distance and the relative rotation, the corresponding Tello commands are issued. The drone first rotates clockwise or counterclockwise by the required angle and then moves forward by the required distance, splitting the movement into multiple commands if the distance exceeds the limitations of a single Tello command. While Drone 2 flies over each defective segment, an alert code is sent to the Arduino to indicate that the vehicle is re-inspecting a faulty region. When all waypoints have been visited, the second drone lands and the mission is concluded.

VI. YOLO TRAINING FOR FIRE, ANOMALIES AND SHOES

A. Problem Definition and Classes

The perception module must detect three types of events that are relevant to pipeline safety. The first type corresponds to fire, where the detector must recognize flames or fire-like patterns near the duct. The second type corresponds to generic anomalies, such as visual defects or foreign objects on or near the duct, for example leaks, stains, suspicious objects or vehicles. The third type corresponds to shoes, where the presence of human footwear or lower legs close to the duct is used as an indicator of human presence in hazardous or unauthorized areas. The custom YOLO detector is trained to recognize these three classes in the visual domain provided by the Tello camera.

B. Dataset Construction

The dataset used for training combines real and augmented images. The real images are captured in the test environment by flying the Tello over the mock pipeline and include scenes without anomalies, scenes with simulated fire such as printed or artificial flames, scenes with various anomalies and scenes with people standing near the duct so that their shoes are visible. Augmented images are generated by applying random rotations, scalings, horizontal flips, brightness and contrast adjustments and perspective transforms to increase robustness to variations in viewpoint, lighting and scale.

C. Annotation Format and Dataset Split

Images are annotated manually using tools compatible with the YOLO format, for example the online platform make-sense.ai, which is what our team used. Each image is associated with a text file where each line encodes one bounding box

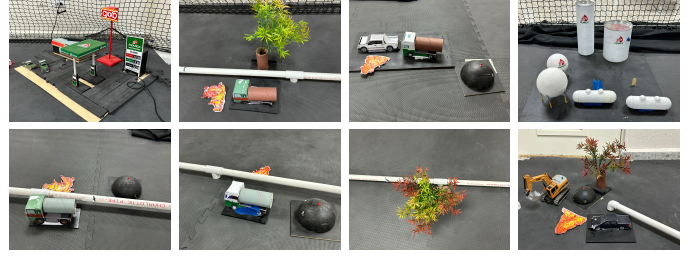


Figure 8: Points of interest from which the YOLO model was trained from.

as a class identifier followed by the normalized coordinates of the box center and its width and height. All coordinates are normalized to the range between zero and one relative to image width and height. The dataset is split into a training set that contains the majority of images, a validation set used to tune hyperparameters such as the number of epochs, learning rate and confidence threshold, and a test set reserved for final performance estimation.

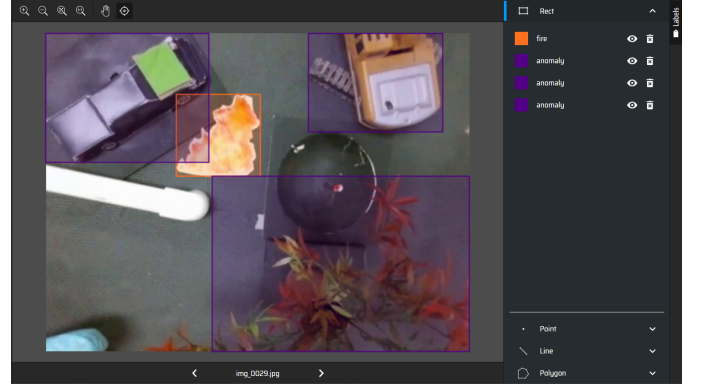


Figure 9: Image labeling interface in makesense.ai.

D. Model Architecture, Training Setup and Evaluation

The model is implemented using the Ultralytics YOLO framework, which provides modern YOLO architectures optimized for real-time detection [3], [4]. A lightweight backbone such as a nano version of YOLOv8 is selected to enable real-time inference on a CPU-only system, the number of classes is set to three and the class names correspond to fire, anomaly and shoe. During training the input resolution is typically set to 640×640 pixels, and during inference it is reduced to 320×320 to lower latency during flight. Training is launched with a command that specifies the dataset configuration file, the base model, image size, number of epochs, batch size and device. After training, the checkpoint that achieves the best validation performance in terms of mean Average Precision is exported as `best.pt`, which is the model loaded in the flight code. Performance is evaluated using standard detection metrics such as precision and recall per class, mean Average Precision at intersection-over-union thresholds and confusion matrices that reveal typical misclassifications. For the present application,

able to see the positive effects of those strategies on our team's workflow. I'm incredibly satisfied with the work our team was able to accomplish and I hope to be involved in more projects like this one in the future.

B. Joel Aragón

Throughout this project, I experienced what it really means to move from theoretical knowledge to real autonomous systems. Working with dual drones, real-time vision, and control algorithms showed me that successful performance is not only about good code, but about patience, testing, and understanding how small details like lighting, tuning, and communication stability can completely change system behavior. This project forced me to think in a more structured way, planning states, transitions, and safety logic instead of just focusing on isolated tasks.

From a personal perspective, this experience strengthened my confidence in working with drones and autonomous platforms. Even with limitations in time and resources, I feel that I gained solid practical skills in control, visual processing, and mission design. Seeing the system operate as a complete solution was very rewarding, and it confirmed my interest in continuing to grow in the areas of robotics, control, and intelligent aerial systems.

C. Emilio Carrillo

Throughout this project I learned how to turn separate concepts, PID control, visual servoing, real-time video processing and YOLO detection, into a complete working dual-UAV system. I better understood how sensitive real robots are to tuning, dataset quality and network reliability, and how important a clear state machine and logging structure are when two drones depend on each other's data. In future work, I would like to improve the robustness of the detector with a larger and more varied dataset, and extend the system to truly cooperative multi-UAV operation with on-board inference, inter-drone communication and basic 3D mapping in more realistic pipeline scenarios.

D. Carlos Cuadras

During this project, I had the opportunity to understand in depth the security challenges that Mexico currently faces regarding fuel pipelines and the ongoing issue of gasoline theft. These situations highlight the urgency of adopting modern technologies capable of improving monitoring, prevention, and rapid response. Unmanned aerial vehicles, in particular, offer valuable advantages because they can operate in dangerous areas, gather real-time information, and assist in decision-making without putting human lives at risk. For these reasons, designing an effective strategy became essential—not only to help reduce gasoline theft within the industry, but also to prevent accidents caused by manual inspections or unsafe conditions around pipeline zones.

In my specific case, I focused on developing programming-based strategies that enable a drone to capture visual and sensor information from its surroundings and make autonomous

decisions based on that data. I also learned how multiple drones equipped with specialized devices can collaborate, each carrying out a specific task such as surveillance, mapping, or hardware inspection. Throughout this process, I strengthened several engineering abilities, including software development, computer vision, control systems, and sensor data acquisition. This project allowed me to integrate these skills into a functional system, reinforcing my understanding of how advanced technologies can be applied to solve real-world industrial problems.

E. Iván Rodríguez

As final conclusions and learnings for this drone academic minor, I have to say that it has been one of the projects where the most has been learned in such a short time, considering that we truly began putting effort into the project during the last 6 weeks of the semester.

I have to say that, personally, I would have liked to be able to learn much more about something related to vision, neural networks, machine learning, deep learning, and all those terms in the concentration. I know that, due to external reasons beyond my control, that could not happen in the best way. However, I believe that the tools we were able to apply for the challenge—such as training a YOLO model, doing programming in Python, a state machine, and implementing PID controllers in a very robust way—both for a drone competition and for the drones implemented in the challenge, have made this a good project that has left a lot of satisfaction in me.

The results speak for themselves: we were able to win first place, and we received congratulations in the final presentation of the challenge. We trust that our model works and will continue working as soon as we can or decide to test it.

And I believe that, in general, I can say that I am capable of carrying out missions with drones. I have a background vast enough to say that I have a specialization in control.

APPENDIX A

TEAM CHARTER (BIO AND INTERESTS)

The team is composed of five Bachelor of Mechatronics Engineering students from Tecnológico de Monterrey, Campus Monterrey: Álvaro Aguirre Castillo, Joel Aragón Balderas, Emilio Carrillo Zardain, Carlos Cuadras Santaella, and Iván Enrique Rodríguez Estrada. All team members share a strong interest in unmanned aerial vehicles, control systems, and applied computer vision. Throughout the degree they have participated in laboratory projects and extracurricular activities involving robotics, embedded systems, and automation, which motivated them to choose a challenge that integrates visual servoing, deep-learning-based perception, and multi-UAV coordination. The dual-UAV inspection project is aligned with their long-term professional goals of working in areas such as autonomous systems, industrial automation, and infrastructure monitoring, and provides an opportunity to gain practical experience in designing, implementing, and validating end-to-end autonomous missions on real hardware.

APPENDIX B
TEAM PROJECT CONTEXT AND SCOPE

A. *Project Name*

Project Title: Autonomous Dual-UAV Inspection of Fuel Pipelines for Fuel Theft Detection.

B. *Theme and Project Charter*

1) *Project Theme:* The project focuses on the development of a visual control and mission-planning framework for two small drones that inspect oil pipelines to identify potential fuel theft events. The mission is divided into two stages:

a) *Fast perimeter inspection:* A first UAV follows a square trajectory around the pipeline perimeter, classifying inspection points as:

- **Positive inspection point:** no truck nor fire detected.
- **Negative inspection point:** clandestine tap or suspicious installation detected (truck or fire).

b) *Focused re-inspection:* After the first mission is complete and the UAV has landed, a second UAV executes a slower trajectory visiting only the negative points and performing a hover of approximately 5 s at each location for detailed inspection and data capture.

The project is framed within the course *Autonomía de vehículos aéreos no tripulados – Control Visual para Drones* and builds upon prior work on square-trajectory autonomous flight and real-time video streaming.

2) *Team Work Rules:*

a) *Attendance and Punctuality:*

- All members attend scheduled meetings (in-person or online) and lab sessions.
- Delays greater than 10 minutes must be notified in the group chat at least 30 minutes in advance.

b) *Responsibilities and Deliverables:*

- Each task in the Gantt chart has a clearly assigned owner.
- Owners are responsible for:
 - Planning and executing the task.
 - Communicating blockers early.
 - Uploading code, documents, and evidence to the shared repository before internal deadlines.

c) *Communication:*

- Main communication channel: agreed group chat (WhatsApp).
- Technical decisions are documented briefly (1–3 lines) in a shared log (e.g., Google Docs/Notion).
- Any change affecting other modules must be communicated the same day.

d) *Version Control and Documentation:*

- Code must be stored in a shared repository (GitHub/shared folder) with clear naming and comments.
- Each functional milestone includes:
 - A short README explaining how to run the code.
 - At least one test video or screenshot when applicable.

e) *Quality and Safety:*

- Indoor flights follow the safety guidelines given by the professors.
- No flight is performed without:
 - Sufficient battery level as defined by the team.
 - Clear take-off and landing zones.

- If safety is compromised, any member can veto a test.

f) *Decision-Making:*

- Technical conflicts are first discussed within the team.
- If no agreement is reached, the issue is escalated to the professor for guidance.
- Priorities for each week are agreed at the start of the week and revisited after every major test.

3) *Penalties for Non-Compliance:* To keep commitments realistic and fair, the following penalties are agreed upon:

a) *Missed Deadline Without Justification:*

- First occurrence: the responsible member must recover the delay and support a teammate's task during the same week.
- Second occurrence: the member's internal evaluation for "responsibility/participation" is reduced by one level (e.g., from "high" to "medium") and they assume an additional documentation or testing task.

b) *Not Attending Meetings or Labs Without Notice:* The absent member must:

- Review all agreements and progress on their own time.
- Prepare a brief written summary of their understanding and next actions.

Repeated absence can be reported to the professor as lack of participation.

c) *Ignoring Safety Rules:*

- Immediate suspension from piloting or running physical tests for at least one session.
- The member must assist with non-flight tasks (logging data, video recording, documentation) during that period.

d) *Persistent Low Contribution:*

- After being warned by the team, if there is no improvement, the case is escalated to the professor, who may adjust the individual grade according to the course policies.

These rules are designed to protect team commitment and ensure that the project can be successfully completed within the semester.

C. *Project Scope / Problem Statement*

1) *Problem Statement:* Fuel theft from oil pipelines is a critical safety, environmental, and economic problem. Traditional inspection methods require human patrols or ground vehicles, which are time-consuming, expensive, and sometimes dangerous. There is a need for an autonomous inspection approach that can:

- Rapidly patrol pipeline segments.
- Detect potential fuel theft indicators such as clandestine taps.

- Distinguish between legitimate PEMEX operations and suspicious activity.
- Provide focused, high-resolution inspection at critical points.

The project addresses this need through a dual-drone inspection architecture tailored to a simplified experimental scenario.

2) Project Scope:

a) In-Scope:

- Implementation of an autonomous square trajectory covering the perimeter of a pipeline segment with Drone 1.
- Visual detection and classification of inspection points into:
 - Positive (no truck nor fire detected).
 - Negative (truck or fire detected).
- Generation of a list (or set of waypoints) corresponding to negative inspection points based on vision outputs or pre-labeled markers.
- Implementation of a second mission where Drone 2:
 - Flies a slower trajectory visiting only the negative points.
 - Performs a hover of approximately 5 seconds at each negative point for detailed inspection.
- Sequential operation: only one drone flies at a time to simplify coordination and safety.
- Integration of:
 - Flight control modules (take-off, landing, square trajectory, waypoint navigation).
 - Visual detection module (YOLO or similar) at least in an offline or semi-online scenario.
 - Basic logging of detections, waypoints, and mission outcomes.

b) Out of Scope:

- Real-time coordination of multiple drones flying simultaneously.
- Large-scale pipeline networks beyond the test environment.
- Full industrial robustness (weather, long-range communication, regulatory constraints).
- Deployment of the system in real PEMEX infrastructure (work remains at lab/prototype scale).

3) *Success Criteria:* The project will be considered successful if, by the end of the semester:

a) Drone 1:

- Takes off, executes a square trajectory around a defined perimeter, and lands autonomously.
- Marks inspection points and differentiates at least two classes (no suspicious activity vs. clandestine tap) in the experimental setup.

b) Drone 2:

- Uses the list of negative inspection points generated by Drone 1.
- Flies to each negative point sequentially.
- Performs a stable hover of approximately 5 s at each negative point before proceeding to the next.

c) Integration and Evidence:

- At least one complete dual-mission run is recorded on video (mission 1 + mission 2).
- A final report documents architecture, algorithms, tests, and results.

D. Updated Gantt Diagram with Responsibilities and Dates

1) Time Horizon:

- Initial diagnosis and early corrections: 13–21 Nov 2025.
- Optimization and technical reporting: 24–28 Nov 2025.
- Final tests and integration: 1–5 Dec 2025.

2) Task Breakdown and Responsibilities:

Table I: Task Breakdown and Responsibilities

ID	Task	Responsible(s)	Start	End
T1	Diagnóstico inicial	Everyone	13/11/25	14/11/25
T2	Correcciones iniciales	Flight Coordinator	14/11/25	17/11/25
T3	Pruebas iniciales	Machine Vision	14/11/25	21/11/25
T4	Reporte técnico #1	Documentarian	18/11/25	21/11/25
T5	Mejora de rutina de despegue	Flight Coordinator	24/11/25	28/11/25
T6	Reporte técnico #2	Documentarian	24/11/25	28/11/25
T7	Pruebas completas	Integrator	01/12/25	04/12/25
T8	Integración cooperativa	Integrator	01/12/25	04/12/25
T9	Reporte técnico #3	Documentarian	04/12/25	05/12/25
T10	Evaluación final del mes	Everyone	04/12/25	05/12/25

3) Gantt Diagram Description:

- The *Diagnóstico inicial* (T1) is completed at the start of the project.
- *Correcciones y Pruebas Iniciales* (T2–T4) span the period from 14 to 21 November.
- *Optimización y Reporte Técnico* (T5–T6) take place from 24 to 28 November.
- *Pruebas Finales e Integración* (T7–T10) are carried out from 1 to 5 December, concluding with the final evaluation.

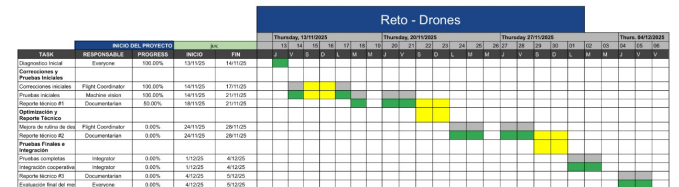


Figure 11: Project Gantt chart for the dual-UAV inspection challenge.

APPENDIX C RISK ANALYSIS

The dual-UAV inspection project involves several technical, operational, and organizational risks. On the technical side, the main risks are related to the reliability of visual perception and the stability of the flight controllers. Line detection can fail under adverse lighting conditions, reflections, or partial

occlusions, which may cause the drone to deviate from the desired path or enter recovery modes more frequently than expected. Similarly, the YOLO detector may produce false positives or false negatives, especially for fire and anomaly classes, which can lead to incorrect labeling of inspection points or unnecessary alarms. There is also a risk associated with wireless communication, as intermittent WiFi connectivity or packet loss may interrupt video streaming or control commands.

From an operational perspective, indoor flights with small drones must comply with the safety rules provided by the instructors. Collisions with obstacles, people, or the environment could damage the hardware or compromise safety. Battery limitations constitute an additional risk, since incomplete missions due to low battery may reduce the available time for testing and validation. Organizationally, the project depends on coordinated work among team members; delays in one module (for example, perception, control, or documentation) can impact integration and final testing.

To mitigate these risks, the team adopted several strategies. Line detection and YOLO thresholds are tuned using offline experiments before being deployed in flight, and the code includes memory modes and elevation search behaviors to recover from temporary perception failures. All flight tests are conducted in controlled indoor environments with clearly delimited take-off and landing zones, and only one drone is airborne at a time. Pre-flight checklists and conservative battery policies are enforced to reduce the likelihood of aborted missions. Finally, tasks are distributed explicitly in the Gantt chart, internal deadlines are set earlier than course deadlines, and regular progress meetings are used to detect and address delays as soon as possible.

APPENDIX D

TECHNICAL SOLUTION FOR THE CHALLENGE

The proposed technical solution is based on a dual-UAV architecture in which each drone has a clearly defined role and both are coordinated through shared data structures rather than simultaneous flight. Drone 1 is responsible for fast patrol and initial detection. It uses a vision-based line tracking module to follow a white line painted on the floor that represents the fuel pipeline, relying on HSV thresholding, contour analysis, and PID visual servoing to remain centered over the duct while progressing along it. A custom-trained Ultralytics YOLO model runs in real time on the ground station, detecting three classes of interest: fire, anomalies, and shoes. When an anomaly detection appears in a region of interest in front of the drone, the mission logic transitions to an inspection state in which the vehicle hovers over the point for several seconds while YOLO continues to search for fire. At the end of each inspection, the point is labeled as safe or faulty depending on whether fire was observed, and the result is stored in a text file.

Drone 2 implements the second stage of the solution: focused re-inspection of faulty points. It reads the results generated by Drone 1 and builds a mask of inspection points

marked as faulty. Using a simple two-dimensional coordinate map of the test environment, the mission planner converts this mask into a sequence of waypoints. For each waypoint, the code computes the relative rotation and distance from the current position, issues the corresponding yaw and forward commands, and optionally performs a brief hover to allow for detailed observation or higher-resolution data capture. By separating fast coverage and detailed re-inspection into two sequential missions, the solution reduces the time spent over benign regions while concentrating flight time and attention on locations where potential fuel theft indicators have been detected.

APPENDIX E

PROGRAMS AND CODE STRUCTURE

The implementation is organized around a main Python script and several supporting configuration and training artifacts. The core mission logic for both drones is implemented in a single script, `reto_final.py`, which contains the video acquisition thread, perception functions for line detection and YOLO inference, the state machine that governs the behavior of Drone 1, and the waypoint-based mission for Drone 2. This script also handles WiFi network switching using `pywifi`, communication with the DJI Tello drones through the `djitellopy` library, and serial communication with the Arduino board for external alerts.

The YOLO detector is trained using the Ultralytics framework. A dataset configuration file (for example, `oil_duct.yaml`) specifies the paths to the training, validation, and test sets, as well as the three class names (fire, anomaly, shoe). A separate training script or notebook launches the `yolo detect train` command with the selected base model, image size, and hyperparameters. The resulting checkpoint with best validation performance is exported as `best.pt`, which is loaded at runtime by `reto_final.py`.

APPENDIX F

RELEVANT FILES

A. Google Drive

For ease of access for all members of the team, we used a shared Google Drive folder where all relevant files to the project (codes, photos, etc.) can be found. The link to view the folder and its contents is the following: Google Drive folder (access with institutional account only).

B. Video evidence

Several videos documenting both our failed tests and functional final runs have been uploaded to YouTube and can be viewed through the following links:

- Successful run #1.
- Successful run #2.
- Failed tests.

C. Presentation

The Canva presentation used during the team's live exhibition can be seen on the following link: presentation.

REFERENCES

- [1] F. Aljaleud, *et al.*, “Autonomous Multi-UAV Path Planning in Pipe Inspection Missions,” *Mathematics*, vol. 11, no. 9, 2023.
- [2] C. Lyu, *et al.*, “UAV-Based Deep Learning Applications for Automated Infrastructure Inspection,” 2025.
- [3] Ultralytics, “Ultralytics YOLO Documentation,” 2023. [Online]. Available: <https://docs.ultralytics.com>
- [4] N. Ladi, “Object Detection with a Pre-Trained Ultralytics YOLOv8 Model,” Ultralytics Blog, 2024.
- [5] D. Fuentes, “DJI TelloPy: A Python Wrapper for the Tello Drone,” 2018. [Online]. Available: <https://github.com/damiafuentes/DJITelloPy>
- [6] S. Roos-Hoefgeest, *et al.*, “A Vision-Based Approach for Unmanned Aerial Vehicles to Track Industrial Pipes for Inspection Tasks,” 2022s.